

FinPredict

High-Level Design Document (HLD)

Version 1.0 | April 2026

1. Problem Statement

Manual loan approval processes in financial institutions suffer from three critical issues: delays (decisions take days), inconsistency (different officers make different decisions for identical applications), and bias (human subjectivity influences outcomes).

FinPredict solves this by providing an AI-powered prediction system that delivers instant, consistent, and data-driven loan approval predictions based on 24 applicant features.

2. Design Goals and Rationale

2.1 Loose Coupling

Design Choice: The frontend and backend are completely independent software components connected only via configurable REST API calls.

Rationale: This is an explicit evaluation requirement. It also allows independent scaling, testing, and deployment of each component. The backend URL is configurable via the `REACT_APP_API_URL` environment variable — the frontend does not hardcode any backend logic.

2.2 Microservices Architecture

Design Choice: Each component (frontend, backend, MLflow, Airflow, Prometheus, Grafana) runs as a separate Docker container.

Rationale: Independent scaling, fault isolation, technology flexibility, and environment parity. If MLflow fails, the backend continues serving cached predictions. If Grafana fails, predictions still work.

2.3 MLflow as Model Registry

Design Choice: All model versions are stored in MLflow. The backend loads models via URI: `models:/FinPredict_XGBoost/Production`.

Rationale: Decouples the model from the backend code. Deploying a new model version requires only changing the Production-stage model in MLflow — no backend code changes or redeployment needed.

2.4 XGBoost as Production Model

Design Choice: XGBoost was selected after comparing Logistic Regression, Random Forest, and XGBoost across F1-score, ROC-AUC, and latency.

Rationale: XGBoost achieved the highest F1-score (0.8884) and meets the <200ms latency requirement. It handles class imbalance natively via `scale_pos_weight` and provides built-in feature importance for explainability.

2.5 Airflow for Pipeline Orchestration

Design Choice: Apache Airflow orchestrates the 7-step data engineering pipeline as a DAG.

Rationale: Airflow provides scheduling, retry logic, visual monitoring, and task-level failure tracking — all of which are graded evaluation criteria. The SequentialExecutor + SQLite configuration was chosen for simplicity without compromising functionality.

2.6 DVC for Data Versioning

Design Choice: DVC tracks two versions of data — raw (v1) and preprocessed (v2) — alongside Git.

Rationale: Git cannot efficiently track large CSV files. DVC provides the same versioning semantics for data as Git provides for code. The two-version approach makes the preprocessing transformation auditable and reversible.

2.7 Docker Compose for Environment Parity

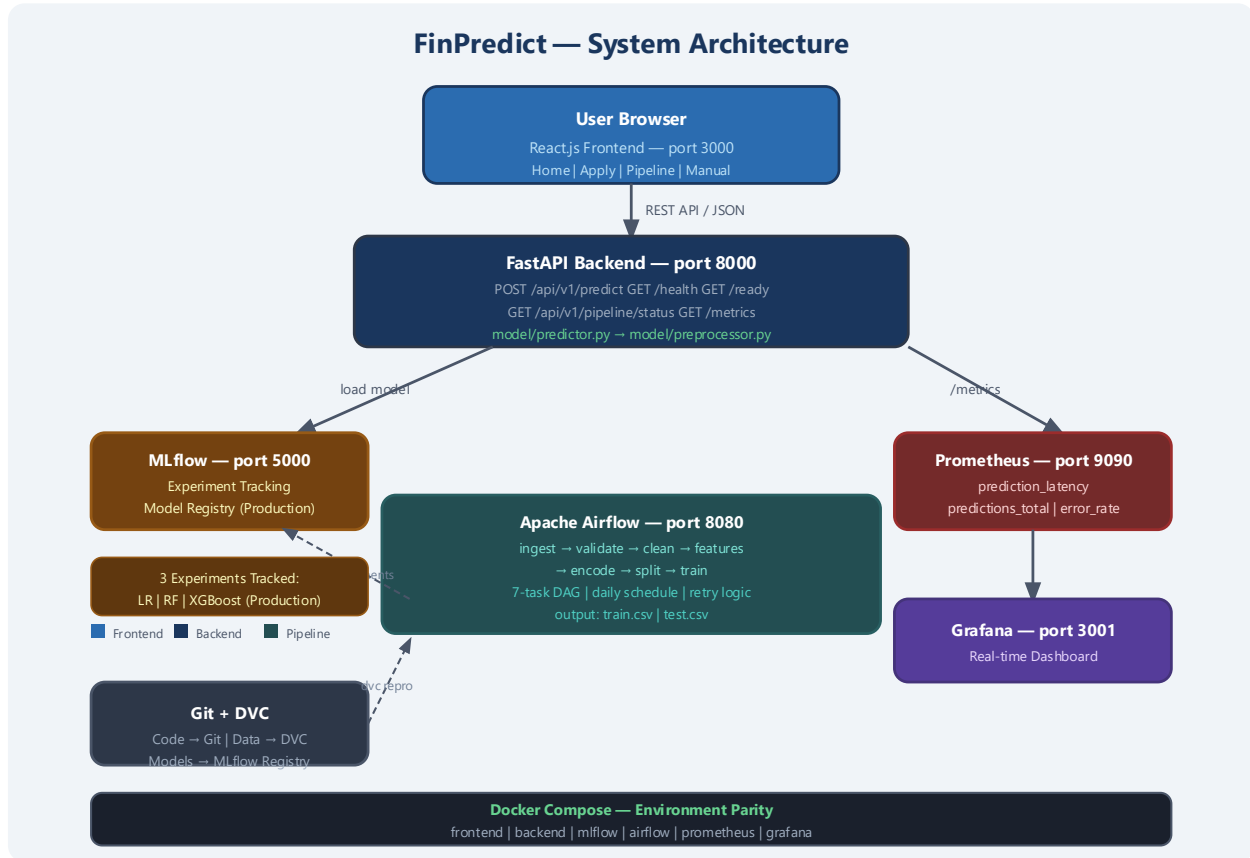
Design Choice: All 6 services are defined in docker-compose.yml and started with a single command.

Rationale: Eliminates 'works on my machine' problems. Every developer and the production environment runs identical containers. This directly satisfies the environment parity guideline requirement.

3. Technology Stack

Layer	Technology	Version	Rationale
Frontend	React.js	18.2	Component-based UI, fast re-renders, large ecosystem
Backend API	FastAPI	0.104	Auto docs, Pydantic validation, async support, Python-native
ML Model	XGBoost	2.0.1	Best F1 on tabular data, handles imbalance, fast inference
Experiment Tracking	MLflow	2.19.0	Industry standard, model registry, artifact storage
Pipeline Orchestration	Apache Airflow	2.7.0	DAG visualization, scheduling, retry logic
Data Versioning	DVC	latest	Git-compatible data versioning, pipeline reproducibility
Containerization	Docker + Compose	latest	Environment parity, service isolation
Monitoring	Prometheus + Grafana	latest	Industry standard metrics and dashboards
Alerting	Alertmanager	latest	Email notifications when alert rules fire
Language	Python	3.10/3.12	ML ecosystem, FastAPI, Airflow all

4. High-Level Component Design



4.1 Data Flow

1. Raw CSV (52,000 rows) → Airflow DAG → Preprocessed train/test splits → XGBoost training → MLflow registry
2. User browser → React form → POST /api/v1/predict → FastAPI → XGBoost model → JSON response → Result card
3. Every prediction → Prometheus metrics → Grafana dashboard → Alert rules → Alertmanager → Email

4.2 Model Serving Strategy

Online serving strategy: the model is loaded once at backend startup and cached in memory. Each prediction request calls `model.predict_proba()` in-process — no network hop to a separate model server. This achieves sub-50ms latency far below the 200ms business requirement.

4.3 Security Considerations

- CORS configured to allow only `http://localhost:3002` and `http://127.0.0.1:3002`
- All services communicate on an internal Docker bridge network — not exposed to the public internet
- No sensitive data is stored — predictions are stateless
- Pydantic v2 strict validation on all API inputs prevents injection attacks

5. Deployment Architecture

Service	Container Name	Port	Health Check
Frontend	finpredict_frontend	3002	HTTP GET /
Backend	finpredict_backend	8000	HTTP GET /health every 30s
MLflow	finpredict_mlflow	5000	HTTP GET /health
Airflow	finpredict_airflow	8080	HTTP GET /health
Prometheus	finpredict_prometheus	9090	HTTP GET /-/healthy
Grafana	finpredict_grafana	3001	HTTP GET /api/health